Lab 01 : E-Commerce Order Analytics Pipeline

Use Case

You are a Data Engineer at an e-commerce company.

Every day:

- Orders are generated
- Data needs to be processed
- Reports must be created for business teams

Instead of manually running scripts, you automate everything using **Apache Airflow**

What You Will Build

A pipeline that:

1. Generates dummy order data
2. Processes the data
3. Stores processed output
4. Logs results

All orchestrated using **Airflow DAG**

Prerequisites

- Ubuntu OS based VM
- Python 3.8+

STEP 1 — Connect to VM

ssh -i astro-key.pem ubuntu@your-public-ip

Update system:

sudo apt update

STEP 2 — Install Python & Dependencies

sudo apt install python3-pip python3-venv -y

Check version:

python3 --version

STEP 3 — Create Virtual Environment

mkdir airflow-project
cd airflow-project

python3 -m venv airflow_venv
source airflow_venv/bin/activate

STEP 4 — Install Airflow

Airflow needs a specific installation method:

export AIRFLOW_VERSION=2.9.1
export PYTHON_VERSION=3.10

pip install "apache-airflow==${AIRFLOW_VERSION}" \
 --constraint "https://raw.githubusercontent.com/apache/airflow/constraints-${AIRFLOW_VERSION}/constraints-${PYTHON_VERSION}.txt"

STEP 5 — Initialize Airflow

Set Airflow home:

export AIRFLOW_HOME=~/airflow

Initialize DB:

airflow db init

Create user:

airflow users create \
  --username admin \
  --firstname Neeraj \
  --lastname User \
  --role Admin \

```
--email admin@example.com \
--password admin
```

STEP 6 — Start Airflow Services

Start scheduler:

airflow scheduler

Open new terminal

Start webserver:

airflow webserver --port 8080

STEP 7 — Access Airflow UI

Open browser:

http://<YOUR-VM-PUBLIC-IP>:8080

STEP 8 — Explore Airflow UI

In the UI, observe:

DAGs Page

- List of workflows

Graph View

- Visual pipeline

Tree View

- Execution history

Task Logs

- Debugging details

Setting up Airflow as a Docker Container

 # Update packages
sudo apt update

 # Install Docker
sudo apt install -y docker.io

 # Start and enable Docker
sudo systemctl start docker
sudo systemctl enable docker

 # Add ubuntu user to docker group (no sudo needed)
sudo usermod -aG docker ubuntu

 # Apply group changes (re-login or run this)
newgrp docker

 # Verify
docker --version

sudo curl -L "https://github.com/docker/compose/releases/latest/download/docker-compose-$(uname -s)-$(uname -m)" \
  -o /usr/local/bin/docker-compose

sudo chmod +x /usr/local/bin/docker-compose

 # Verify
docker-compose --version

Create Airflow Directory:
mkdir ~/airflow-docker && cd ~/airflow-docker
mkdir -p ./dags ./logs ./plugins ./config

Download Official docker-compose.yaml
curl -LfO 'https://airflow.apache.org/docs/apache-airflow/2.9.1/docker-compose.yaml'

Set Airflow UID
echo -e "AIRFLOW_UID=$(id -u)" > .env
cat .env # Should show: AIRFLOW_UID=1000

Initialize Airflow Database
docker-compose up airflow-init

Start All Airflow Services
docker-compose up -d

Verify All Containers are Running
docker-compose ps

STEP 9 — Run Example DAG

Airflow comes with default DAGs.

Enable:

`example_bash_operator`

Click:

- Toggle ON
- Click ▶️ Trigger DAG

Observe:

- Graph view
- Task execution

STEP 10 — Create Real-World DAG

Create DAG file:

cd ~/airflow/dags
nano ecommerce_order_pipeline.py

Paste This Code

"""
DAG: ecommerce_order_pipeline
Description: Optimized E-Commerce Order Processing Pipeline
- Generates orders (simulated / pluggable source)
- Validates and processes orders via XCom
- Saves a final sales report to disk
Schedule: Daily
"""

import json
import logging
import os
from datetime import datetime, timedelta

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.exceptions import AirflowFailException

# ── Logger ──────────────────────────────────────────────────────────────────
log = logging.getLogger(__name__)

# ── Default Args ────────────────────────────────────────────────────────────
default_args = {
    "owner": "data-team",
    "depends_on_past": False,
    "retries": 3,
    "retry_delay": timedelta(minutes=5),
    "execution_timeout": timedelta(minutes=30),
    "email_on_failure": False,   # Set True + configure SMTP to get alerts
    "email_on_retry": False,
}

# ── Report output path (easy to swap for S3 / GCS later) ────────────────────
REPORT_DIR = "/tmp/airflow_reports"
REPORT_PATH = os.path.join(REPORT_DIR, "sales_report.json")


#
# ────────────────────────────────────────────────────────────────────────────
# Task 1 – Generate Orders
# Push raw orders to XCom so no inter-task file dependency exists.
# Replace the static list with a DB query or API call in production.
#
# ────────────────────────────────────────────────────────────────────────────
def generate_orders(ti, **context):
    log.info("Generating orders for execution date: %s", context["ds"])

    # ── Simulated source  (swap with DB / API call) ──────────────────────
    orders = [
        {"order_id": 1, "customer": "Alice",   "amount": 250, "status": "completed"},
        {"order_id": 2, "customer": "Bob",     "amount": 450, "status": "completed"},
        {"order_id": 3, "customer": "Charlie", "amount": 300, "status": "pending"},
        {"order_id": 4, "customer": "Diana",   "amount": 0,   "status": "cancelled"},
    ]
```

```
    # ── Validate schema ──────────────────────────────────────────────────────
    required_keys = {"order_id", "customer", "amount", "status"}
    for order in orders:
        if not required_keys.issubset(order.keys()):
            raise AirflowFailException(
                f"Order {order} is missing required fields: {required_keys}"
            )
        if not isinstance(order["amount"], (int, float)) or order["amount"] < 0:
            raise AirflowFailException(
                f"Order {order['order_id']} has invalid amount: {order['amount']}"
            )

    log.info("Generated and validated %d orders.", len(orders))

    # ── Push to XCom (no temp files needed) ──────────────────────────────────
    ti.xcom_push(key="raw_orders", value=orders)


#
# ─────────────────────────────────────────────────────────────────────────────
# Task 2 – Process Orders
# Pull from XCom, filter & aggregate, push result back to XCom.
#
# ─────────────────────────────────────────────────────────────────────────────
def process_orders(ti, **context):
    orders = ti.xcom_pull(task_ids="generate_orders", key="raw_orders")

    if not orders:
        raise AirflowFailException("No orders received from generate_orders task.")

    log.info("Processing %d orders...", len(orders))

    # ── Filter: only completed orders count toward revenue ──────────────────
    completed = [o for o in orders if o["status"] == "completed"]
    pending   = [o for o in orders if o["status"] == "pending"]
    cancelled = [o for o in orders if o["status"] == "cancelled"]

    total_sales     = sum(o["amount"] for o in completed)
    average_order   = total_sales / len(completed) if completed else 0

    summary = {
        "execution_date":   context["ds"],
        "total_orders":     len(orders),
        "completed_orders": len(completed),
```

```
        "pending_orders":   len(pending),
        "cancelled_orders": len(cancelled),
        "total_sales":      round(total_sales, 2),
        "average_order":    round(average_order, 2),
        "top_order":        max(completed, key=lambda o: o["amount"]) if completed else None,
    }

    log.info("Order summary: %s", json.dumps(summary, indent=2))

    # ── Push processed summary to XCom ──────────────────────────────────
    ti.xcom_push(key="order_summary", value=summary)


# ─────────────────────────────────────────────────────────────────────────────
# Task 3 – Save Report
# Pull summary from XCom and persist to disk (or S3 / DB in production).
# ─────────────────────────────────────────────────────────────────────────────
def save_report(ti, **context):
    summary = ti.xcom_pull(task_ids="process_orders", key="order_summary")

    if not summary:
        raise AirflowFailException("No order summary found. Cannot save report.")

    # ── Ensure output directory exists ──────────────────────────────────
    os.makedirs(REPORT_DIR, exist_ok=True)

    # ── Write report with execution-date stamp ──────────────────────────
    dated_path = os.path.join(
        REPORT_DIR, f"sales_report_{context['ds']}.json"
    )
    with open(dated_path, "w") as f:
        json.dump(summary, f, indent=2)

    log.info("Report saved to: %s", dated_path)
    log.info("Total Sales: $%.2f across %d completed orders.",
             summary["total_sales"], summary["completed_orders"])

    # ── Alert if no sales were recorded ─────────────────────────────────
    if summary["total_sales"] == 0:
        log.warning("Total sales is $0 for %s — check order source!", context["ds"])
```

```
#
# DAG Definition
#
with DAG(
    dag_id="ecommerce_order_pipeline",
    description="Daily e-commerce order ingestion, processing, and reporting",
    default_args=default_args,
    start_date=datetime(2024, 1, 1),
    schedule="@daily",          # replaces deprecated schedule_interval
    catchup=False,
    max_active_runs=1,          # prevent overlapping runs
    tags=["ecommerce", "etl", "sales"],
    params={                    # runtime-overridable via Airflow UI / API
        "min_order_amount": 0,
    },
) as dag:

    t1_generate = PythonOperator(
        task_id="generate_orders",
        python_callable=generate_orders,
    )

    t2_process = PythonOperator(
        task_id="process_orders",
        python_callable=process_orders,
    )

    t3_report = PythonOperator(
        task_id="save_report",
        python_callable=save_report,
    )

    # ── Pipeline flow
    t1_generate >> t2_process >> t3_report
```

STEP 11 — Refresh Airflow UI

Go to UI
You will see:

ecommerce_order_pipeline

Enable it ✅

STEP 12 — Run Your DAG

Click:

- Trigger DAG ▶️

STEP 13 — Inspect DAG Execution Flow

Graph View

You will see:

generate_orders → process_orders → save_report

Task States

State	Meaning
success	Task completed
running	In progress
failed	Error

Logs

Click any task → View Logs

Example:

Orders generated!
Total Sales: 1000

Report saved successfully!

STEP 14 — Validate Output (EC2)

cat /tmp/orders.json
cat /tmp/processed_orders.json